Computergrafik und Computeranimation

Arnd Vitzthum

2. 2D-Vektorgrafik

…

2.6 Linienstile, dicke Linien und Antialiasing

2.7 Füllen von Polygonen

…

Bilder z.T. aus. Klawonn, Grundkurs Computergrafik mit Java, Vieweg+Teubner, 2010

Grundlegende Methoden, Pixelnachbarschaft

- Pixelbasiertes Füllen
  - Füllen nach Scan Conversion (Rasterung)
  - Verwendung in rasterbasierten Zeichenprogrammen
- Polygonbasiertes Füllen
  - Füllen während Scan Conversion → effizienter
  - Verwendung in vektorbasierten Programmen

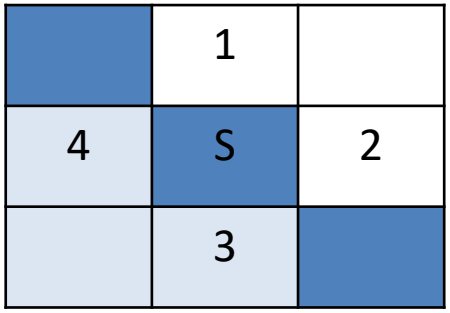
- Pixelnachbarschaften eines Pixels S
  - 4 Pixel
  - 8 Pixel

	1	
4	S	2
	3	

8	1	2
7	S	3
6	5	4

Flächendefinition durch Randpixel

- Betrachte 8 benachbarte Pixel für Pixelverbindungen
- Folgt man von einem Startpixel ausgehend den Verbindungen, erreicht man den Startpixel wieder → Randlinie der eingeschlossene Fläche
- Betrachtet man nur 4 benachbarte Pixel, werden Pixelverbindungen u. U. ausgelassen (siehe Beispiel unten)



Flächendefinition durch Innenbereichspixel

- Betrachte 4 benachbarte Pixel
- Beginne mit einem Pixel und folge (rekursiv) den vier benachbarten Pixeln
- Alle überschrittenen Pixel definieren Innenbereich einer Fläche
- Betrachtet man alle 8 benachbarten Pixel eines Pixels wird die Randlinie u. U. “übersprungen” → keine klare Abgrenzung des Innenbereichs (siehe Beispiel unten)
- **Wichtig für pixelbasierte Füllalgorithmen!**

8	1	2
7	S	3
6	5	4

Pixelbasiertes Füllen

- Boundary-Fill
  - Beginne mit Startpixel (Seed) innerhalb des zu füllenden Bereichs
  - Rekursives Füllen der Pixel, solang die Farbe des zu füllenden Pixels *nicht der Rand- oder der Füllfarbe entspricht*
- **Füllen von mehrfarbigen Flächen möglich, wenn definierte Randfarbe vorhanden**

- Probleme
  - Zeit- und Speicherbedarf (Stapelüberlaufgefahr) beim Füllen großer Flächen

→ Optimierung durch Begrenzung d. Zahl rekursiver Aufrufe

  - Zeilenweises füllen

Pixelbasiertes Füllen

- Flood-Fill
  - Beginne mit Startpixel (Seed) innerhalb des zu füllenden Bereichs
  - Rekursives Füllen der Pixel, solang die Farbe des zu füllenden Pixels der ursprünglichen Innenbereichsfarbe entspricht
- **Füllen von Flächen mit mehrfarbigem Rand möglich**

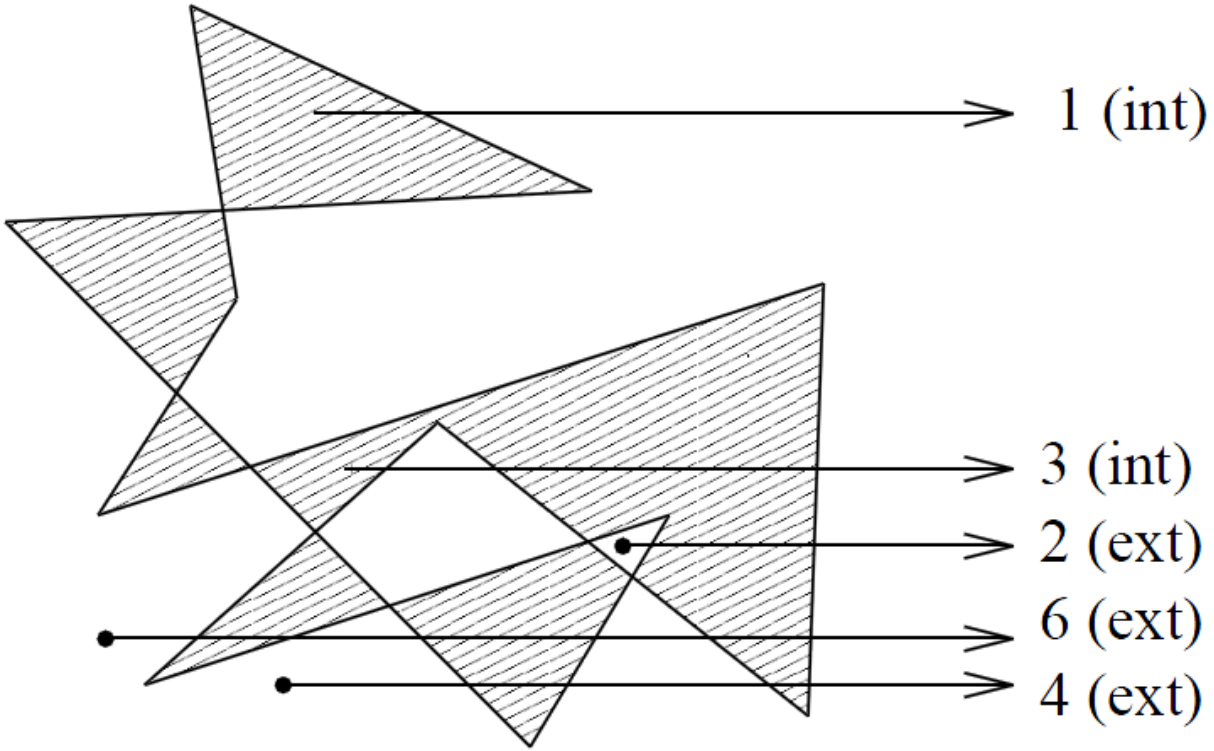
- Probleme und Optimierung wie bei Boundary-Fill

Polygonbasierte Verfahren – Scan Line – Odd-Parity-Regel

- Setze Pixel, wenn innerhalb des Polygons
- Wann ist ein Pixel oder Punkt innerhalb des Polygons?

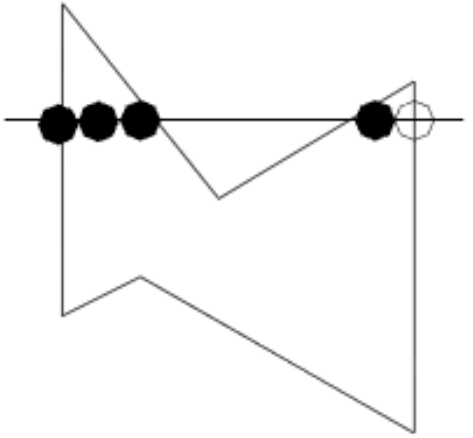
- ➢ Betrachtung eines Punkts P innerhalb des Polygons
- ➢ Bewegung entlang einer von P ausgehenden Halbgeraden h
- ➢ Zähle überquerte Schnittpunkte von h mit den Polygonrändern
- ➢ Die von einem Punkt *innerhalb des Polygons* ausgehende Halbgerade weist immer eine ungerade Schnittpunktanzahl auf
- ➢ Vgl. Even-Odd-Regel in Java2D

Odd-Parity-Regel



Polygonbasierte Verfahren – Scan Line

- Aufwendig: Betrachtung einer Halbgeraden für jedes Pixel
- Stattdessen: Bewegung entlang der Pixelzeilen (Scan Lines) und Bestimmung innerer und äußerer Pixel einer Zeile
- Dabei automatisch Sortierung der Schnittpunkte Scan-Line/Polygonzug in Reihenfolge aufsteigender x-Koordinaten
  - → $x_1 < \ldots < x_n$ sind x-Koordinaten der Schnittpunkte
  - → Färbe Pixel von x_1 bis x_2, x_3 bis x_4, usw.
- Schnittpunktzahl je Scan-Line im Normalfall gerade!



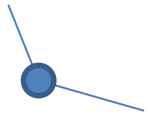
Scan Line Verfahren – Sonderfälle

- *Horizontale* Kanten: Ignorieren

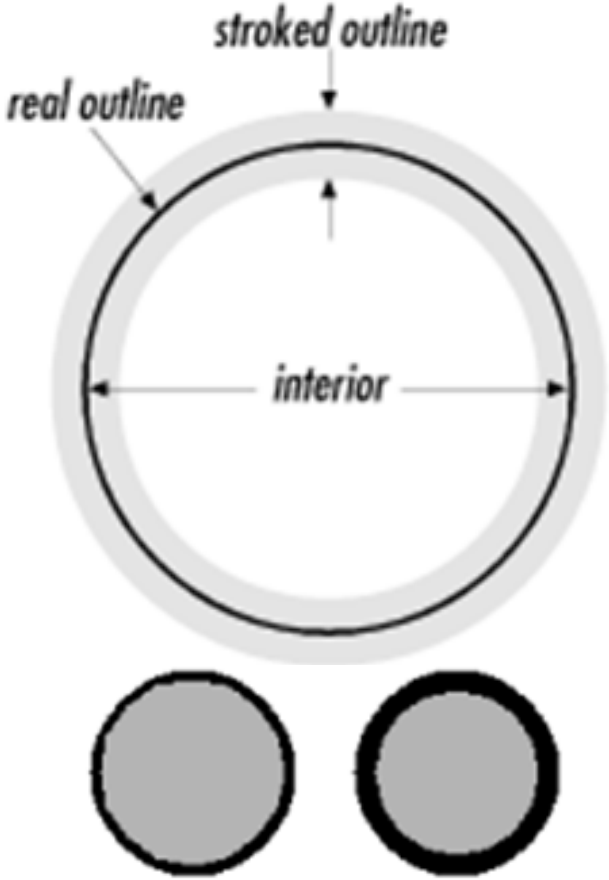
- Scan-Line schneidet Eckpunkt
  - Eckpunkt ist lokales Minimum od. Maximum
    - Punkte auf den zwei im Eckpunkt zusammentreffenden Kanten weisen in unmittelbarer Umgebung des Eckpunktes keine niedrigeren bzw. höheren y-Koordinaten auf
    - Erzeugung von zwei Schnittpunkten



  - Andernfalls: Nur ein Schnittpunkt je Eckpunkt



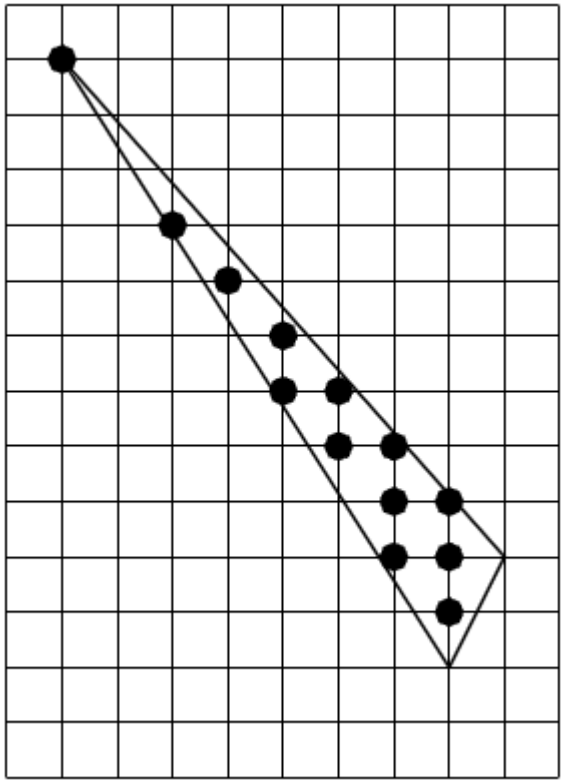
Scan Line Verfahren – Kontur und Füllung



- Bei breiten Rändern überschneiden sich Randpixel und innere Pixel
- Unterschiedliche Zeichenreihenfolge → unterschiedliche Ergebnisse
- Ähnliches “Problem” bei Antialiasing des Randes

Bilder: Knudsen, Java 2D Graphics, O'Reilly 1999

Scan Line Verfahren - Probleme



- Beim Füllen „schmaler” Polygone können Aliasingeffekte auftreten → Zusammenhängende Fläche wird durch getrennte Mengen von Pixeln dargestellt

Füllen mit Texturen

- Farbinformation für zu zeichnende Pixel wird aus einem rechteckigen Rasterbild (der Textur) extrahiert
- Position der Textur innerhalb der zu füllenden Fläche wird durch Ankerpunkt festgelegt
- Ist die Textur zu klein, um die gesamte Fläche auszufüllen, kann die Textur kachelartig wiederholt werden
- Bei Vorgabe fester Kachelgröße muss die Textur auf die Kachelgröße skaliert werden



Glättung bei Texturübergängen

0.1	0.1	0.1
0.1	0.2	0.1
0.1	0.1	0.1

- Einsatz einer Gewichtsmatrix
  - Die neue Farbe eines Pixels wird als gewichtete Summe der eigenen Farbe und den Farben der Nachbarpixel bestimmt
  - Die Farbe des aktuellen Pixels wird mit dem „zentralen“ Element der Matrix gewichtet
  - Auf die Farben der Nachbarpixel werden entsprechend „benachbarte“ Matrixelemente angewendet
- Bei Texturen empfiehlt sich Glättung der Randpixel
  - „weiche“ Kachelübergänge
  - Als Nachbarpixel eines Randpixels werden die entsprechenden Pixel des gegenüberliegenden Randes angesehen
  - Beispiel: Pixel am rechten Rand der Textur → rechtes Nachbarpixel ist Pixel am linken Texturrand in derselben Zeile